

I've seen e-commerce companies lose millions when their product search couldn't handle common typos and language shortcuts that shoppers use.

How I tested this

I built a testing framework that examines how embedding models respond to different text variations. Using cosine similarity as the metric (where 1.0 means identical meaning and 0.0 means completely unrelated), I ran hundreds of test cases.

I tested multiple models including MSMarco DistilBERT, OpenAI text embedding, and many others. The patterns were similar across most transformer-based embeddings.

The critical flaws I uncovered

Capitalisation blindness

This one shocked me. Embedding models see “Apple announced new products” and “apple announced new products” as EXACTLY the same — perfect 1.0 similarity score.

I ran into this with a product catalogue system. The search couldn't distinguish between the brand ‘Apple’ and the fruit ‘apple’. Customers searching for Apple products got recipes for apple pie. Think your customers would be happy with that? I wouldn't be.

Think about cases where capitalisation changes meaning — ‘Polish’ vs ‘polish’, ‘March’ vs ‘march’, ‘Bill’ vs ‘bill’. For legal or medical text, these distinctions can be crucial. We're basically working with models that are partially blind to written language.

There are ways to fix these issues, which I'll cover later.

But first, let's see what else is broken. Note that sometimes this could be helpful even if these differences don't matter for your use case.

Numerical amnesia

This one floored me too. Embedding models see “The investment returned

2% annually” and “The investment returned 20% annually” as practically the same thing — a crazy high 0.97 similarity score.

I encountered this with a financial document search. The algorithm couldn't tell the difference between ‘management fee: 0.2%’ and ‘management fee: 2.0%’. Investors looking for low-fee funds were recommended expensive options. Would you want your retirement account making that mistake? I sure wouldn't.

Why does this matter? Think about cases where numerical values are critical — dosage instructions, engineering tolerances, financial returns, and contract deadlines. For investment or medical text, these distinctions can be life changing. We're working with models that are numerically illiterate despite

handling text full of important quantities.

Negation blindness

This one's actually dangerous. Adding ‘not’ to a sentence — literally flipping its meaning — barely affects similarity scores. We routinely saw scores above 0.95 for complete opposites.

“The treatment improved patient outcomes” vs “The treatment did not improve patient outcomes” had a 0.96 similarity. When I showed this to a doctor using our medical search system, he was horrified. He physically backed away from the computer. And he was right to be scared. We were building a system doctors would use to find treatment protocols. Get this wrong, and people could die.

Negation isn't some edge case — it's fundamental to human language. When your system can't tell ‘effective’ from ‘ineffective’ or ‘safe’ from ‘unsafe’,

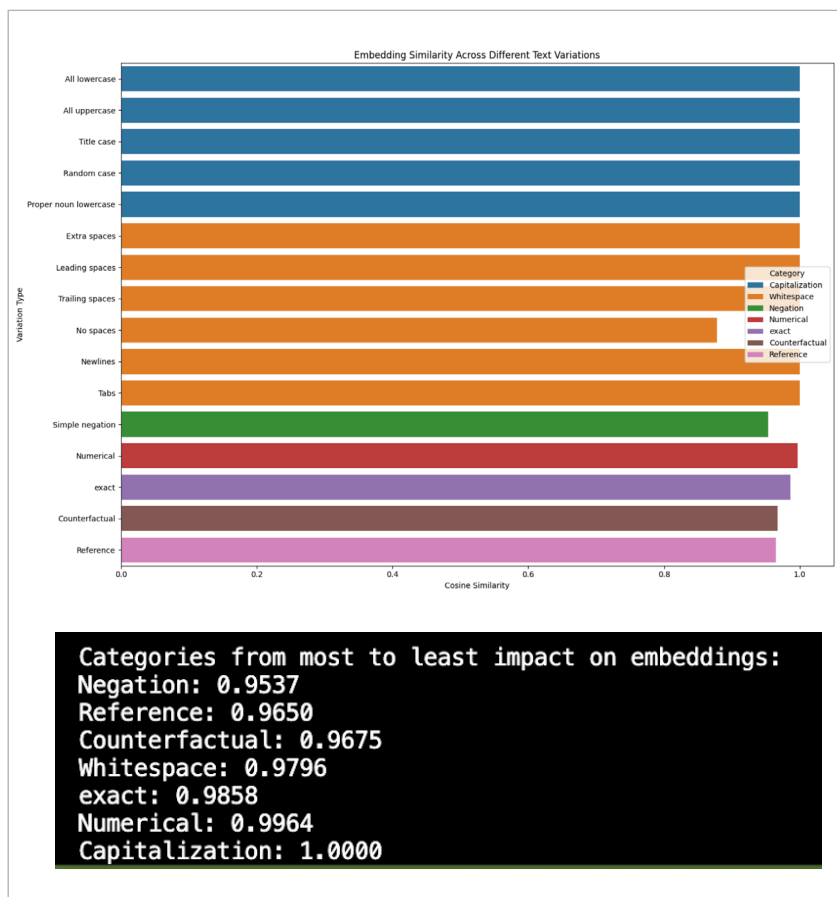


Figure 1: msmarco-distilbert-base-tas-b embedding score across different test cases